HTTP Streaming

chunked transfer, backpressure and gRPC streams

Watch the 4-min video

In one line: instead of building a huge response in memory, send it in pieces as you produce it (chunked transfer in HTTP/1.1, data frames in HTTP/2), read the data lazily, and let TCP backpressure keep memory flat.

1 Why stream, and how it works on the wire

A user clicks **Export** and asks for five million orders as a CSV file. If the server builds the whole file in memory first, it may need gigabytes of RAM. The user stares at a spinner for a minute, or the server simply runs out of memory and crashes. The fix is **streaming**: send the file in pieces while you are still producing it.

In **HTTP/1.1** a server normally sends a `Content-Length` header, so it must know the body size before it sends the first byte. With `Transfer-Encoding: chunked` it doesn't. The body goes out as a series of chunks; each one starts with its size in hexadecimal, and a **zero-length chunk** marks the end.

HTTP/2 and HTTP/3 don't use chunked encoding at all (it is not allowed there). A body is already a series of **DATA frames**, and the last one carries an end-of-stream flag. Same idea, built into the protocol.

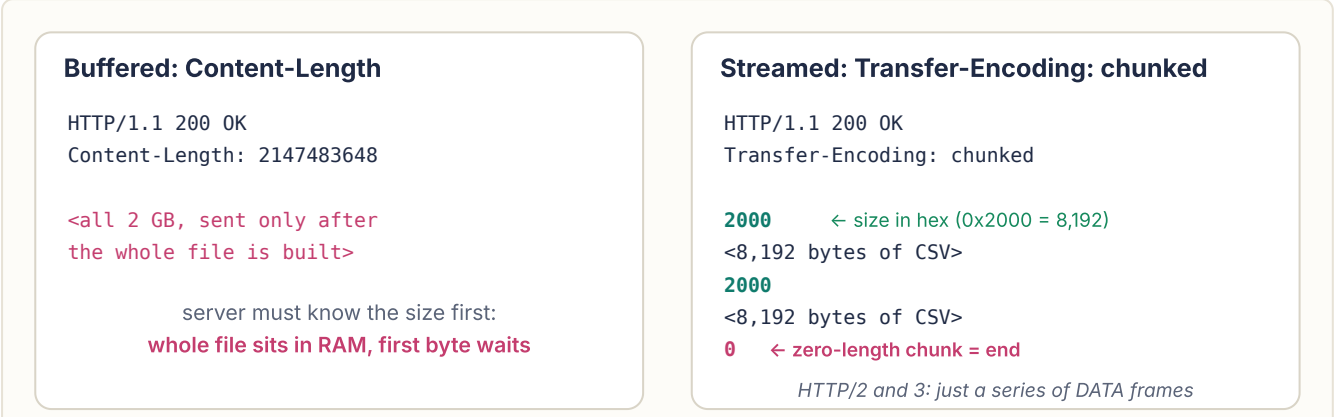


Figure 1. Left: with Content-Length the server must finish the whole body first. Right: chunked encoding sends each piece with its size, and a 0 chunk ends the body.

So the server can start sending before it has finished computing: the **first bytes leave in milliseconds**. **Memory stays flat**, because only one chunk is held at a time. And the client can **process data as it arrives**.

2 Example: a streaming CSV export in FastAPI

Read rows with a **server-side cursor**, in batches of about 1,000, so the database driver never loads the full result. A **generator** turns each batch into CSV lines and yields them. Return a `StreamingResponse` with media type `text/csv` and a `Content-Disposition` header so the browser saves it as a file.

WORKED EXAMPLE

```
async def rows(): async for batch in cursor_batches(size=1000): yield to_csv(batch)
return StreamingResponse(rows(), media_type="text/csv", headers={"Content-Disposition":
"attachment; filename=orders.csv"})
```

Five million rows at about 100 bytes each is roughly **500 MB** of CSV. Buffered, that is 500 MB or more in RAM per export; ten users at once is 5 GB. Streamed, each export holds one batch: 1,000 × 100 bytes ≈ **100 KB**, and the download starts immediately. (SQLAlchemy: `stream_results=True` + `yield_per(1000)`; psycopg: a named cursor.)

3 Backpressure: when the client reads slowly

What if the client downloads slowly, say on a weak mobile link? **TCP flow control** handles it. The client's receive window fills, then the server's send buffer fills. Once that buffer is full, the server's write call waits. In an async framework, the generator simply isn't asked for the next chunk until there is room.

So a slow reader automatically slows down the producer, instead of data piling up in memory. This only works if you produce lazily: a generator that builds a list of every row first has already lost.

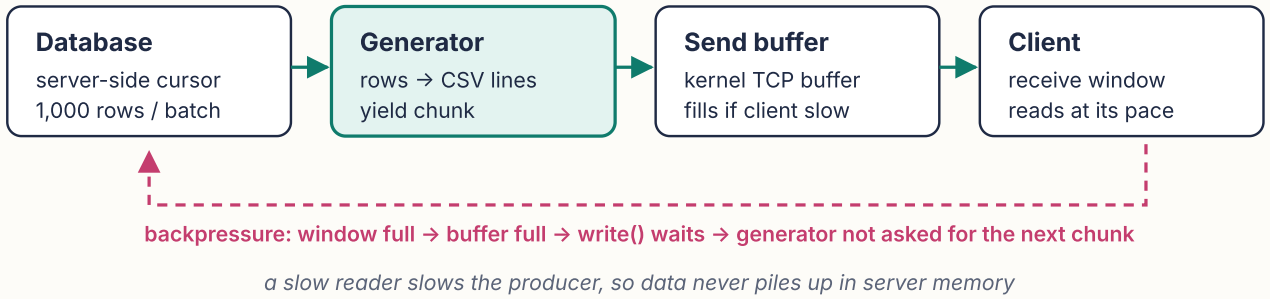


Figure 2. Data flows left to right; pressure flows back right to left. Each stage only pulls the next piece when the stage after it has room.

4 gRPC: streaming as a first-class idea

gRPC runs over HTTP/2 and builds streaming into its contract. A method in the `.proto` file can mark either side as a `stream`, giving four call types:

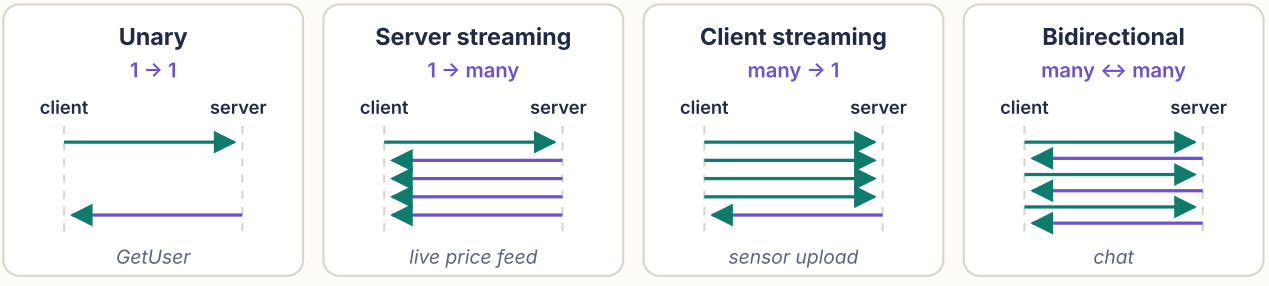


Figure 3. Black arrows are client messages, purple are server messages. In bidirectional mode both sides send independently, at any time.

5 Pitfalls

- **Buffering proxies.** A proxy may hold the whole response before forwarding it. In NGINX set `proxy_buffering off;` or send `X-Accel-Buffering: no`.

- **Errors mid-stream.** The status `200` was sent with the first chunk, so a failure halfway cannot change it. Add an error marker or a row count / checksum at the end. (gRPC sends its status in HTTP/2 trailers, after the data, so it handles this.)
- **Timeouts.** Set idle and total timeouts so a stuck client can't hold a connection open forever.

COMMON MISTAKES

- Using a `StreamingResponse` but fetching all rows first with `fetchall()`: memory is still huge.
- Testing locally (works) and deploying behind a buffering proxy (download starts only at the end).
- Assuming a 200 means the file is complete.

6 Key terms

Chunked transfer: body as size-prefixed chunks

DATA frame: HTTP/2 and 3 unit of body data

Backpressure: slow consumer slows the producer

Content-Length: body size, sent before the body

Server-side cursor: rows fetched in batches

Trailers: headers sent after the body

7 Interview questions

Q1. How do you stream a large HTTP response? BASIC

Send it in pieces as you produce it: `Transfer-Encoding: chunked` in HTTP/1.1, or DATA frames in HTTP/2 and 3. Yield rows lazily from a server-side cursor, so memory stays at one batch, not the whole file.

Q2. What does chunked transfer encoding look like on the wire? BASIC

Each chunk is its size in hex, CRLF, the bytes, CRLF. For example `2000` then 8,192 bytes. A chunk of size `0` ends the body, optionally followed by trailers. No Content-Length is sent.

Q3. What streaming modes does gRPC support? BASIC

Four: unary (1 request, 1 response), server streaming (1 request, many responses), client streaming (many requests, 1 response) and bidirectional streaming (both sides stream at once).

Q4. Streaming works locally, but in production the download starts only after 60 s. Why? SCENARIO

Something in between is buffering the whole response, usually NGINX with `proxy_buffering on` (the default) or a compression layer. Turn buffering off for that route or send `X-Accel-Buffering: no`.

Q5. A 1 Mbit/s client downloads a 500 MB export. Does server memory grow? SCENARIO

No, if the producer is lazy. TCP flow control fills the client's window and the server's send buffer, then the write waits and the generator pauses. But it takes 500 × 8 / 1 ≈ 4,000 s (over an hour), so set a total timeout.

Q6. The database fails at row 3 million of a streamed export. How does the client find out? TRICKY

The status 200 is already sent, so it can't change. Close the connection abnormally (in HTTP/1.1, no final 0 chunk; in HTTP/2, a stream reset) and/or write an explicit error marker, and include a row count or checksum at the end so clients can detect a truncated file.

Q7. Why can't you use Transfer-Encoding: chunked with HTTP/2? TRICKY

HTTP/2 forbids it: the body is already split into DATA frames with their own lengths, and the end-of-stream flag marks the end. Proxies convert between the two.

Next: SD-N-027 · Forward and reverse proxies backendroadmap.com